

LECTURE 01 STUDENT NOTES

DevOps Culture through a Failed Release

Traditional delivery, Agile roots, continuous delivery, and the role of cloud

CSC418 - DevOps for Cloud Computing

Syllabus focus: Traditional development vs DevOps culture and the role of cloud

In this lecture You will use Verona Chapter 1 to examine a failed release, compare traditional handoffs with DevOps culture, and explain how cloud capabilities can make feedback arrive earlier.

Write in the spaces provided. The questions are part of the notes, not a separate test.

1. What you should be able to explain

By the end of the lecture, check whether you can:

- Compare traditional handoffs with shared responsibility for a service.
- Explain how Agile collaboration influenced the DevOps movement.
- Distinguish useful delivery speed from faster execution of waste.
- Connect cloud capabilities to earlier testing and feedback.

2. Case study: the Friday release

Situation The admissions portal must be updated before applications open on Monday. Development sends the release to operations late on Friday. QA tested an older configuration. The team has one maintenance window left.

Time	What happens	What becomes difficult
4:30 PM	Development emails a ZIP file and a deployment document.	Operations sees the change for the first time.
6:00 PM	QA confirms that testing used a different configuration.	The test result does not fully represent production.
9:10 PM	The application cannot connect to the production database.	The missing environment variable is found very late.
10:20 PM	Operations completes a rollback.	Students still depend on the previous version.

Before reading further, write one sentence: What is the main problem in this story?

3. Traditional delivery and DevOps culture

Traditional delivery often divides work into stages owned by separate departments. A request moves from planning to development, testing, approval, deployment, and operation. Each handoff can add waiting and can remove context from the change.

DevOps joins development and operations with testing, security, product, and other contributors around one service. Specialists keep their expertise, but they share information and responsibility for the result experienced by the user.

Traditional handoff pattern	DevOps culture
Each role sees only its part of the change.	Contributors can see the change and its current evidence.
Testing and operations join near the end.	Testing and operational needs influence the change early.
Configuration knowledge lives in documents or memory.	Configuration and repeatable steps are versioned and checked.
Success means completing a departmental task.	Success means a useful and reliable service outcome.

Case connection Development completed the code, QA completed a test, and operations followed a procedure. Students still received no updated service. Local completion did not produce system success.

4. Where the DevOps idea came from

Verona places DevOps inside the wider Agile movement. The Agile Manifesto, published in 2001, gives greater weight to individuals and interactions than to processes and tools. DevOps applies this collaboration problem across development and operations.

The term grew around the first DevOpsDays event, organized by Patrick Debois in Ghent in 2009. The history matters because it shows that DevOps began as a response to divided work, not as the name of one product or job title.

Milestone	Meaning for this lecture
Agile Manifesto, 2001	Collaboration and working software matter more than rigid handoffs.
DevOpsDays, 2009	The development-operations divide became a named community problem.
DevOps practice	Technical delivery work and cultural change must support each other.

5. Tools should help people interact

Chapter 1 uses a simple example: development, QA, and operations may maintain separate systems for features, bugs, and deployment requests. The tools reflect the organizational walls. A shared workflow can provide one change record with different views for each role.

Tool choice	Question to ask
Separate trackers and request queues	Does information disappear when work crosses a team boundary?
Shared change record with role-specific views	Can every contributor see the current decision and evidence?
Automated repeatable checks	Does the result reach the person who can act on it?

Study point A tool earns its place when it reduces friction, removes repetitive work, or returns evidence sooner. Buying a tool alone does not change ownership.

6. How fast is fast

Verona connects DevOps speed to business value. Faster execution has little value when the work does not help a user or improve a decision. Useful speed removes waiting and returns evidence while a change is still easy to correct.

Faster waste	Useful delivery speed
Approve the same large batch more quickly.	Review a small working change with stakeholders earlier.
Run a fragile manual procedure faster.	Replace repetitive steps with checks that return clear results.
Increase output without learning.	Use evidence to decide what to change next.

In the Friday case, the most expensive wait is the delay before the team tests the real configuration. Faster typing or a shorter meeting would not solve that feedback problem.

7. The Agile wheel of wheels

Agile work runs at several cadences. A developer may edit, build, and test within minutes. Kanban work may be reviewed daily. A Scrum sprint usually lasts two to four weeks. Program and portfolio decisions run over longer periods.

Cadence	Typical work	Evidence DevOps should provide
Minutes	Edit, build, and test	Whether a small code or configuration change works
Day	Operational work and Kanban flow	Where work is blocked and which service needs attention
Weeks	Scrum sprint	A working increment that stakeholders can inspect
Months	Program and portfolio planning	Delivery and service outcomes that support investment decisions

Case connection An eight-week handoff ignores the short feedback loops. DevOps strengthens the longer planning cycles by making technical evidence available sooner.

8. Cargo cult DevOps

Chapter 1 warns about copying the visible form of Agile or DevOps while missing its purpose. A daily stand-up does not create collaboration when teams still hide information. A pipeline does not improve delivery when only one group understands it or when users wait just as long.

Visible practice	Goal test
Daily stand-up	Does information cross the boundary that caused the delay?
Automation platform	Does it remove repetitive work and return useful evidence?
Deployment pipeline	Does it help more than one group and improve the service outcome?
Retrospective	Did the discussion produce a change that prevents recurrence?

Question to remember Who receives useful evidence sooner, and what decision improves?

9. Where cloud computing helps

Cloud services can make infrastructure available through self-service interfaces and APIs. For the admissions team, this could make it practical to create a temporary test environment that resembles production, run checks, and remove the environment afterward.

Cloud capability	How a student should connect it to DevOps	Risk that still needs control
On-demand resources	Create a test environment when it is needed.	Unused resources and unexpected cost
APIs	Describe and repeat environment setup.	Incorrect automation or excessive privilege
Elastic capacity	Test how the portal behaves when demand changes.	Consumption without suitable limits
Measured service	See resource use and relate it to the service.	Collecting data without anyone acting on it

Remember Cloud can shorten the path to useful feedback. It does not automatically create collaboration, good tests, security, or cost control.

10. DevOps and ITIL

Verona explains that DevOps and ITIL can coexist. Small, frequent changes benefit from automated checks, traceable evidence, repeatable deployment, and fast recovery. A large legacy change may still need formal coordination because it has many dependencies and a larger failure impact.

DevOps contribution	How it can support change control
Versioned change and configuration	Reviewers can see exactly what will change.
Automated test and policy results	Approvals can use current evidence instead of verbal assurance.
Small changes and rollback plans	The change is easier to understand and recover.
Monitoring and incident evidence	The organization can confirm the result and learn from failure.

Key distinction Governance should match the risk of the change. DevOps can improve the evidence used by formal controls rather than removing those controls.

11. How would we know delivery is improving?

Question	Evidence the team could collect
Are changes reaching a deployable state sooner?	Lead time for a change
Can the team deliver smaller changes more often?	Deployment frequency and change size
Are changes causing fewer production problems?	Failed deployments, rollbacks, and urgent corrections
Can the team restore the service sooner?	Time from service impact to restoration
Is the team learning from incidents?	Repeated failures and new preventive checks

These measures describe the delivery system. They should help the team decide what to improve, not rank individual students, developers, testers, or operators.

12. In-class worksheet: redesign the admissions release

Full scenario The team releases every eight weeks. Development sends a ZIP file and deployment document to operations. Production configuration is maintained manually. QA receives a representative test environment late. The release needs a two-hour change meeting and a Saturday maintenance window. A missing environment variable causes a 70-minute rollback. A similar failure happened two releases earlier.

12.1 Record the facts

List facts from the scenario without suggesting solutions.

12.2 Explain the causes

Choose three important constraints. Use the lecture ideas, not product names.

Constraint	People / work / technology	Evidence from the scenario

12.3 Propose improvements

For each change, state what feedback will arrive earlier or what risk will be reduced.

Proposed change	Earlier feedback or reduced risk

12.4 Choose evidence

What would you measure during the next few releases to see whether your proposal worked?

13. Review without looking back

1. Why can each department complete its task while the service outcome still fails?
2. How did Agile ideas influence the DevOps movement?
3. What distinguishes useful speed from faster execution of waste?
4. How do short technical loops support longer Agile planning cycles?
5. Why is a pipeline alone weak evidence that a team practices DevOps?
6. Name one cloud capability that can shorten feedback and one risk it introduces.
7. How can DevOps evidence support formal change control?
8. Choose two measures that would help the admissions team judge its next release.

14. Terms to remember

Term	Meaning
Agile cadence	The repeating time interval used to plan, execute, or review work.
Batch size	The amount of change grouped before integration, testing, or release.
Cargo cult	Copying a visible practice while missing the purpose that makes it useful.
Continuous delivery	Keeping software in a state where the team can release it safely and repeatedly.
Deployment	Installing a change in an environment.
Feedback loop	A path that returns useful evidence to people who can act on it.
Handoff	A transfer of work or responsibility between people or teams.
ITIL	A service management framework that formalizes many parts of the service lifecycle.
Lead time	The elapsed time for a change to move between defined start and end points.
Release	Making a deployed capability available to its intended users.
Shared ownership	Responsibility across roles for the service outcome.

15. Chapter 1 reading guide

The syllabus assigns Verona Chapter 1. Use the guide below to read with a purpose. The printed page number appears at the bottom of each book page.

Read	PDF pages	Focus question
Printed pages 1-3	20-22	How do collaboration and tools work together in DevOps?
Printed pages 3-5	22-24	Which waiting time prevents useful value or feedback?
Printed pages 6-7	25-26	Does a visible practice achieve its intended goal?
Printed pages 7-8	26-27	How can DevOps and ITIL coexist in a large organization?

Course file Books/practical-devops.pdf

16. Supporting reading

Use these sources only when you need the specific explanation named here.

- DORA Continuous Delivery: use this page for small batches, fast feedback, and keeping software releasable. <https://dora.dev/capabilities/continuous-delivery/>
- NIST SP 800-145, pages 2-3: use these pages for the cloud characteristics connected to the temporary test environment. <https://doi.org/10.6028/NIST.SP.800-145>
- CSC418 DevOps Syllabus V3.2, page 1: use this page to confirm the Lecture 1 topic and the assigned Verona chapter.

17. Before Lecture 2

Choose one cloud service you already use. Write its name, then decide whether it mainly provides infrastructure, a platform for building applications, or a complete application.

Cloud service	IaaS / PaaS / SaaS?	What the provider manages	What the customer still manages

What became easier because of this service model?

What control, visibility, skill, or portability might the customer give up?

Bring to class Your example does not need to be perfect. We will use it to compare responsibility boundaries in Lecture 2.

18. One-minute exit reflection

Name one traditional delivery constraint from the admissions case and one DevOps response supported by Chapter 1. Explain the connection in two or three sentences.
